

Investigates the Impact of AI-generated Code Tools on Software Readability Code Quality Factor

Hamed Fawareh
Dept. of Software Engineering
Zarqa University, Jordan
fawareh@zu.edu.jo

Hazim M. Al-Shdaifat
Dept. of Software Engineering
Zarqa University, Jordan
hazimshdaifat@gmail.com

Al-Refai, M
Dept. of Software Engineering
Zarqa University,
Jordan
refai@zu.edu.jo

Faid AlNoor Fawareh
Dept. Of Mathmitical Science
Zarqa University, Jordan
Faidalnoor21@gmail.com

Mohammed Khouj
Dept. of Electrical Engineering
University of Business and Technology, Jeddah, 21432,
Saudi Arabia

Abstract—Artificial Intelligence (AI) has made great strides in various industries, including software development, with tools such as ChatGPT changing the way code is written, maintained and developed. This study investigates the impact of AI-generated code on software quality, focusing on readability, code clarity, complexity, and documentation. Comparing AI-generated code with open-source code from GitHub for three tasks of varying difficulty (easy, medium, and hard), we evaluated key metrics such as cyclomatic complexity (CC), code lines, comment density, code-to-code ratio, and variable names. The results revealed that although both AI-generated and human-generated code can be concise and efficient, code generated by AI tools shows a greater focus on documentation more clearly. AI-generated code especially in complex tasks showed improved readability due to higher comment density and the use of clear variable names. These results show that AI tools such as ChatGPT can enhance code quality, especially in the software industry where readability and comprehensibility are critical.

Keyword—Readability, AI-Tools, Code Quality, Software development, ChatGPT

I. INTRODUCTION

Artificial intelligence (AI) has been one of the fastest growing fields; its inception is traced back to the mid-20th century, with early applications in problem-solving and logical reasoning. In these many years, AI has infiltrated every level of industry, from health and finance to manufacturing, with the introduction of machine learning, natural language processing, and autonomous systems [1]. Such growing ability of AI to imitate cognitive functions and perform tasks that used to be the sole realm of human intelligence has led it to be integrated into many aspects of everyday life, changing both consumer experiences and professional practices. The area of software development is where AI has had a lot of breakthroughs with tools for making work more productive and code quality better. Prominent examples are Open AI's ChatGPT and GitHub Copilot, the latter developed together with Open AI. It is in this context that conversational AI, best represented by ChatGPT, supports developers by generating code snippets, making suggestions and providing explanations, and debugging tips. As much as it offers real-time code suggestions, GitHub Copilot also generates full functions based on natural language

descriptions [2]. Another tool is Google Gemini, which enriches the user experience in terms of information retrieval and contextual recommendation [3]. Such tools are revolutionizing software development by automating repetitive works, providing help on demand and making a cooperative environment between human developers and the AI doable. They not only accelerate the development process but also help keep code consistent and of high quality. Code quality is a primary factor in software development and is often appraised on account of its attributes: readability, maintainability, efficiency, and accuracy. Attributes assure that the software created is reliable, user-friendly, and easy to update or debug. Readability: This can be defined as how easily a developer may understand the program for the very important quality of collaboration and future maintainability. AI tools such as ChatGPT and GitHub Copilot improve readability to a great extent by providing proper naming conventions that are clear and consistent, besides organizing code in a more humanly readable fashion [4].

II. RESEARCH BACKGROUND

Tao Zhang focused on the study of deep learning technologies for code summarization and search to detail the capturing of semantic relations in contextual information. The main task of this article was to improve efficiency and maintainability in software development. These are tools that involve deep learning to assist in addressing some of the limitations found in earlier traditional methods, which otherwise seem blind to the structural information at the time. When these AI tools were put into general practice for any engineering tasks, this study had massive improvement to show in what was being carried out. This method was set to reduce developers' workload since it could be a source of more accurate and efficient automation tools [5].

Devalla discussed the ways in which AI could be used to fully automate the different stages of the software development lifecycle, such as coding processes, requirements gathering, and software testing. Their study proved quite beneficial in terms of increases in efficiency, accurateness, and maintainability by the use of machine learning, natural language processing, and deep

learning. The AI technologies provided may not specify the programming languages, but there appears to be a general emphasis on them. Researchers discussed that the approaches driven by AI understand better than others and “refine” the needs of users for software quality. In fact, the comprehensive analysis had details of the deep ability of AI to totally alter the process of software development [1][6].

Perera investigated the effect of code commenting on software quality by focusing on readability and maintainability. Moreover, this study showed some of the areas that create automatic comments, which are consistency, classification, and quality rating. This research was conclusive documentation that improved collaboration and made future efforts of maintenance manageable. With regard to readability and maintainability, the study established that there were substantial roles played by comments in the total quality of a software application [7].

In a series of experiments, Russell A. Poldrack showed that GPT-4 was able to do the code generation. Their experiment also concluded that although it is evident that GPT-4 can bring up significant improvements in code efficiency and accuracy, enormous amounts of human validation are still required. They found out that the refactoring abilities of GPT-4 provided an increase in code quality metrics and that their tests covered greatly, despite many failed applied tests. The research underscored how powerful yet imperfect the current AI-assist coding tools were and how important it was to have human oversight to ensure the correctness of the code. It gave important insights into how AI is integrated into coding tasks [8].

Kazemitabaar explored the impact of AI code generators, specifically OpenAI Codex, on novice programmers in introductory programming courses. They conducted a controlled experiment with 69 participants aged 10-17, who worked on Python code-authoring tasks with and without access to Codex. The study found that using Codex significantly improved code-authoring performance and did not negatively impact subsequent manual code-modification tasks participants who used Codex performed slightly better in retention post-tests. The research highlighted the potential of AI tools to enhance learning and code quality for novice programmers [9].

Antonio Vitale gave another approach to automatically enhance code readability, based on deep learning and, more specifically, the T5 model. A methodology was developed to identify readability-improving commits, and a dataset of a large volume, 122,000 commits, was produced, available from GitHub. They report that training T5 on emulating developer actions for readability enhancement yields perfect prediction accuracy in a range of 21% to 28%. Manual analysis concluded that a significant readability improvement could be achieved for most of the cases without introducing any change to the code behavior manually. This paper proposed a comprehensive approach to automate readability improvement in the code [10].

Burak Yetis, Tiren have compared AI-assisted code generation technologies like GitHub Copilot, Amazon Code Whisperer, and ChatGPT using the HumanEval Dataset. They measured code quality measures such as validity, correctness, security, dependability, and maintainability. The results are such that the code generated by ChatGPT was proper 65.2% of the time, while with GitHub Copilot and Amazon Code Whisperer, it was 46.3% and 31.1%, respectively. This allows practitioners

to gain a deeper understanding of the strengths and weaknesses of the tools. The findings emphasized that the proper tool should be used to improve code quality [2].

Qazi studied AI tools used in software quality assurance followed by software companies from Pakistan. Such studies emphasized benefits available from AI with regards to optimization of QA strategy and business processes. This is despite only 30% of such companies currently applying AI-based tools, a scenario where major challenges were identified and solutions proposed in improving SQA with modern machine learning models. It described the important role played by AI in enhancing software development as well as quality assurance processes [11].

Valentina Piantadosi mined several software repositories on aspects related to the evolution of code readability. Its main goal was to comprehend how code readability evolves in complex software systems and to what extent this aspect concerns developers. The study observed that readability rarely changes during software evolution; files created unreadable rarely become readable, and vice-versa. The study also found that there is a positive correlation between some personal attributes of developers, such as the attention network that leads to code readability, the research work has underpinned the use of readability throughout the lifecycle of software development [12].

R. Poldrack et al. conducted experiments using GPT-4 to generate and refactor computer code. The current study tries to illustrate the true host of capabilities regarding AI-assisted coding tools, such as GPT-4, in improving code quality and readability. From the results of the experiments, it emerges that AI code generation is indeed powerful, but much human validation is still needed in order to ensure the accuracy of its output. Moreover, they found that refactoring with GPT-4 significantly improved code according to established metrics for code quality and hence provided some evidence that AI tools can help in software maintainability [8].

III. APPROACH TAKEN

The approach taken in this paper first started by using ChatGPT as a primary programming assistant for code generation. The programming languages to be used were determined depending on their popularity in the open-source community and their suitability for the tasks at hand such tools and languages are essential for code generation and subsequent code evaluation. Once the coding tasks are identified, ChatGPT generates code for identified tasks with three levels of difficulty (Easy, Medium and Complicated) [13]. The generated AI code is compared to the open-source code developed by human developers. The second approach represents establishing a reliable comparison of open-source code from GitHub repositories carefully selected. The Selected code represents a range of programming tasks classified into three distinct levels of difficulty. This categorization assures a complete evaluation of ChatGPT performance across different levels of difficulty (Easy, Medium and Hard). The comparison between ChatGPT code and open-source code based on readability quality factor which includes these matrices, Cyclomatic Complexity (CC), Lines of Code (LOC), Comment Density, Code-to-Comment Ratio, Variable Names, comparison reveals the effects of AI tools on the readability of code quality. Three types of case

studies on the readability quality factors of the code, in the following section we will discuss the result.

A. Readability Metric

To understand the readability of the cases used in the study, several metrics were used that evaluate different aspects of the code to how easy and simple it is to read and understand the source code of a software program. Common metrics used to evaluate readability include:

1) Cyclomatic Complexity (CC)

This metric calculates the number of linearly independent paths through the source code of a program. A higher level of complexity indicates harder-to-maintain and more complex code [14].

Formula:

$$CC = E - N + 2 \times P$$

Where:

- E is the number of edges
- N is the number of nodes
- P is the number of connected components in the control flow graph

2) Lines of Code (LOC)

A simple but informative metric that measures the total number of lines in a program. Although not a direct indicator of quality, a higher number of lines of code often indicates higher maintenance complexity [15].

Formula: Simply the number of all non-null and non-empty lines of code in a software system.

3) Comments Density

This metric indicates the proportion of the code base that consists of comments that help explain the code. A higher percentage of comments generally enhances readability and maintainability, especially for developers unfamiliar with the details of the project [16-17].

Formula:

$$\text{Comments percentage} = \left(\frac{\text{Number of comment lines}}{\text{Total code lines}} \right) \times 100$$

4) Code-to-Comment Ratio

This metric compares the number of lines of code to the number of comment lines.

Formula:

$$\text{Code-to-Comment Ratio} = \frac{\text{Lines of Code}}{\text{Lines of Comment}}$$

Giving insight into the balance between code and documentation.

5) Variable Names

Descriptive variable names improve code readability. This metric evaluates the use of meaningful and well-chosen variable names, which can significantly reduce the cognitive load required to understand code. This can be evaluated qualitatively or by analyzing name patterns in large datasets.

IV. RESULT AND DISCUSSION

This study tested the readability factor of the quality of code, two types of code first type as code generated by ChatGPT which includes three test cases (Easy, Medium and Complicated). The second type of code open-source code available on GitHub includes three different test cases as well. The result analysis is based on five metrics: Cyclomatic Complexity (CC), Lines of Code (LOC), Comment Density, Code-to-Comment Ratio, and Variable Names.

A. CASE STUDY #1: TWO SUM (EASY)

Applying the five metrics to the code generated by ChatGPT and open-source code [18-19], the results are shown in Table 1. The results of our experiment are as follows:

- Cyclomatic Complexity (CC): Both GitHub and ChatGPT generated code with a CC of 3, indicating a similar complexity.
- Lines of Code (LOC): ChatGPT generated a more concise solution with 11 lines, compared to GitHub's 15 lines.
- Comment Density: GitHub had no comments (0%), while ChatGPT's code had a comment density of 31.25%, reflecting a more interpretive approach.
- Code-to-Comment Ratio: GitHub's lack of comments made it difficult to calculate the ratio, while ChatGPT's ratio was 2.2, indicating balanced documentation.
- Variable Names: Both GitHub and ChatGPT used clear and descriptive variable names, which improve code readability.

TABLE 1: TWO SUM (EASY)

Metrics	GitHub	ChatGPT
Cyclomatic Complexity (CC)	3	3
Lines of Code (LOC)	15	11
Comment Density	0%	31.25%
Code-to-Comment Ratio	-	2.2
Variable Names	Clear	Descriptive and clear

B. CASE STUDY #2: INTEGER TO ROMAN (MEDIUM)

In this case study, we apply average complexity code compared to Case Study #1 as shown in table 2, then we apply

the five metrics to the code generated by ChatGPT and open-source code [20]. The results of our experiment are as follows:

- Cyclomatic Complexity (CC): Both codes had a CC of 3, indicating similarity in complexity.
- Lines of Code (LOC): The number of lines in the GitHub code was 10, while the ChatGPT code was shorter at 9 lines, making the ratio 1.11:1 for ChatGPT.
- Comment Density: The comment density in the GitHub code was 0%, while the comment density in the ChatGPT code was 45.45%, which improves the understandability of the code.
- Code-to-Comment Ratio: The code-to-comment ratio in ChatGPT was 1.8, indicating a balance between code and explanation.
- Variable Names: The names of GitHub variables were not clear, using labels like "cs" and "ans," while ChatGPT provided more descriptive names, improving code readability.

TABLE 2: INTEGER TO ROMAN (MEDIUM)

Metrics	GitHub	ChatGPT
Cyclomatic Complexity (CC)	3	3
Lines of Code (LOC)	10	9
Comment Density	0%	45.45%
Code-to-Comment Ratio	-	1.8
Variable Names	cs, ans, unclear	Descriptive and clear

C. CASE STUDY #3: REGULAR EXPRESSION MATCHING (HARD)

In this case study, we apply more complex code than in Case Studies #1 and #2. Then, we apply the five metrics to the code generated by ChatGPT and open-source code [21]. The results are shown in Table 3, and the outcomes of our experiment are as follows:

- Cyclomatic Complexity (CC): ChatGPT-generated code exhibited higher complexity (CC = 8) compared to GitHub's (CC = 6), indicating more intricate logic in the AI-generated solution.
- Lines of Code (LOC): GitHub's code was more concise with 21 lines, while ChatGPT's solution used 25 lines, making GitHub's code 1.19 times shorter.
- Comment Density: GitHub again had no comments, while ChatGPT included 21.88% comments, improving comprehension for future developers.
- Code-to-Comment Ratio: ChatGPT's code had a higher ratio of 3.57, again highlighting its focus on documentation.
- Variable Names: GitHub used vague names like "f," "s," and "p," while ChatGPT used more descriptive and clear variable names, making its code easier to understand.

TABLE 3: REGULAR EXPRESSION MATCHING (HARD)

Metrics	GitHub	ChatGPT
Cyclomatic Complexity (CC)	6	8
Lines of Code (LOC)	21	25
Comment Density	0%	21.88%
Code-to-Comment Ratio	-	3.57
Variable Names	f, s, p, unclear	Descriptive and clear

V. CONCLUSION

This paper discusses the comparative performance of AI generated code using ChatGPT versus human-generated code from a variety of open-source GitHub repositories on several measures of code quality and readability. The results show that while ChatGPT-generated code is more detailed, it consistently improves readability and maintainability, especially for more complex tasks. Both AI-generated code and human-generated code exhibited slightly higher cyclic complexity, suggesting that AI-generated code can be concise and clear. However, for more difficult tasks, ChatGPT's high comment density and clear variable names significantly improved readability, making it more suitable for applications that require long term maintenance and collaboration. The presence of detailed comments in ChatGPT-generated code is valuable for future updates and teamwork, demonstrating the potential of AI to simplify software development processes.

REFERENCES

- [1] Y. K. Dwivedi, L. Hughes, E. Ismagilova, G. Aarts, C. Coombs, T. Crick, Y. Duan, R. Dwivedi, J. Edwards, A. Eirug, et al., "Artificial intelligence (ai): Multidisciplinary perspectives on emerging challenges, opportunities, and agenda for research, practice and policy," *International journal of information management*, vol. 57, p. 101994, 2021.
- [2] B. Yetis, I. Ozsoy, M. Ayerdem, and E. T. "uz" un, "Evaluating the" code quality of ai-assisted code generation tools: An empirical study on github copilot, amazon codewhisperer, and chatgpt," *arXiv preprint arXiv:2304.10778*, 2023.
- [3] H. R. Saeidnia, "Welcome to the gemini era: Google deepmind and the information industry," *Library Hi Tech News*, no. ahead-of-print, 2023.
- [4] P. J. Guo, "Six opportunities for scientists and engineers to learn programming using ai tools such as chatgpt," *Computing in Science & Engineering*, vol. 25, no. 3, pp. 73–78, 2023.
- [5] T. Zhang, X. Sun, Z. Zheng, and G. Li, "Intelligent analysis for software data: research and applications," *Frontiers of Information Technology & Electronic Engineering*, vol. 23, no. 5, pp. 661–663, 2022.
- [6] Editorial Board. (2024). *Crafting an AI Strategy for Government Leaders: Does your Agency have a Holistic AI Strategy?*. Hikama, 8(4), 191-190, doi: 10.31430/IUTQ6756
- [7] D. Perera, H. Premathilake, K. Thathsarani, R. Nethmini, D. De Silva, and H. Samarasekara, "Analyzing the impact of code commenting on software quality," in *2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pp. 1–6, IEEE, 2023.
- [8] R. A. Poldrack, T. Lu, and G. Begus, "Ai-assisted coding: Experiments" with gpt-4," *arXiv preprint arXiv:2304.13187*, 2023.
- [9] M. Kazemitabaar, J. Chow, C. K. T. Ma, B. J. Ericson, D. Weintrop, and T. Grossman, "Studying the effect of ai code generators on supporting novice learners in introductory programming," in *Proceedings of the 2023*

- CHI Conference on Human Factors in Computing Systems, pp. 1– 23, 2023.
- [10] A. Vitale, V. Piantadosi, S. Scalabrino, and R. Oliveto, “Using deep learning to automatically improve code readability,” in 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 573–584, IEEE, 2023.
 - [11] S. Qazi, M. S. Memon, A. Ali, and S. Nizamani, “Role of artificial intelligence (ai) tools for assuring quality in software,” *Journal of Southwest Jiaotong University*, vol. 57, no. 2, 2022.
 - [12] V. Piantadosi, “On the evolution of code readability,” in 2022 IEEE International Conference on Software Maintenance and Evolution (ICSME), pp. 597–601, IEEE, 2022.
 - [13] Abozaid, A. (2023). A Comparative Theoretical Study of the Dynamics of Security Architecture in the Gulf Regional System since the Arab Uprisings (2011-2021). *Journal of the social sciences*, 51 (1), 9-34
 - [14] A.-J. Molnar and S. Motogna, “A study of maintainability in evolving open-source software,” in *Evaluation of Novel Approaches to Software Engineering: 15th International Conference, ENASE 2020, Prague, Czech Republic, May 5–6, 2020, Revised Selected Papers 15*, pp. 261–282, Springer, 2021.
 - [15] .L. Ardito, R. Coppola, L. Barbato, and D. Verga, “A tool-based perspective on software code maintainability metrics: A systematic literature review,” *Scientific Programming*, vol. 2020, no. 1, p. 8840389, 2020.
 - [16] T. Hericko and B. Sumak, “Exploring maintainability index variants for software maintainability measurement in object-oriented systems,” *Applied Sciences*, vol. 13, no. 5, p. 2972, 2023.
 - [17] 262588213843476, “Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target. You may assume that each input would have exactly one solution, and you may not use the same element twice. You can return the answer in any order. Example 1: Input: nums = [2,7,11,15], target = 9 Output: [0,1] Output: Because nums[0] + nums[1] == 9, we return [0, 1]. — gist.github.com.”
 - [18] <https://gist.github.com/ktareq24/48502d692ac0e40e644d3226a2b9519df> ileetcode-java-1-two-sum-java. [Accessed 19-09-2024].
 - [19] “leetcode/solution/0000-0099/0012.Integer to Roman/README EN.md at main · doocs/leetcode — github.com.”
 - [20] <https://github.com/doocs/leetcode/blob/main/solution/00000099/0012.Integer> [Accessed 13-09-2024].
 - [21] “leetcode/solution/0000-0099/0010.Regular Expression Matching/README EN.md at main · doocs/leetcode — github.com.” <https://github.com/doocs/leetcode/blob/main/solution/00000099/0010.Regular> [Accessed 19-09-2024].